

PANDUAN PEMBELAJARAN

BAHASA PEMROGRAMAN C++

Dari dasar hingga proyek mini • Edisi pemula

ISI BUKU

Konsep dasar • Variabel • Operator • Percabangan • Perulangan • Fungsi • Array & vector • Class • Pointer • STL • File • Exception • Template • Praktik baik • Proyek mini

Diterbitkan oleh
Theport Studio

Buku pengantar praktis dengan penjelasan dan contoh kode C++ yang dapat dikompilasi.

1. Mengetahui C++ dan Cara Kerjanya

C++ adalah bahasa pemrograman berperforma tinggi yang mendukung pemrograman prosedural, berorientasi objek, dan generik. Program C++ ditulis sebagai teks sumber, lalu compiler menerjemahkannya menjadi program yang dapat dijalankan. Contoh compiler populer ialah GCC, Clang, dan MSVC. Standar modern yang umum dipakai mencakup C++17, C++20, dan C++23.

Alur dasarnya: tulis berkas berakhiran .cpp, kompilasi, perbaiki pesan kesalahan, lalu jalankan hasilnya. Pilih standar konsisten, misalnya C++17, dan biasakan mengaktifkan peringatan compiler agar masalah terdeteksi lebih awal.

Contoh kode

```
// Simpan sebagai halo.cpp
#include <iostream>

int main() {
    std::cout << "Halo, C++!\n";
    return 0;
}
// GCC/Clang: g++ -std=c++17 -Wall -Wextra halo.cpp -o halo
// Jalankan: ./halo
```

Catatan penting: Program dimulai dari fungsi main. Header iostream menyediakan fasilitas input/output. Simbol :: pada std::cout menunjukkan anggota namespace std. Nilai balik 0 menandakan program selesai dengan normal.

2. Struktur Program dan Komentar

Baris `#include` meminta compiler menyertakan deklarasi pustaka. Fungsi adalah blok bernama yang melakukan tugas; kurung kurawal membatasi blok, sedangkan titik koma mengakhiri banyak pernyataan. C++ peka terhadap huruf besar-kecil: `nilai` dan `Nilai` adalah nama berbeda.

Komentar membantu menjelaskan alasan dan maksud kode, bukan mengulang hal yang sudah jelas. Komentar satu baris memakai `//`; komentar beberapa baris memakai `/* ... */`. Komentar tidak dieksekusi.

Contoh kode

```
#include <iostream>

// Fungsi utama program
int main() {
    /* Teks berikut dicetak
       ke terminal pengguna. */
    std::cout << "Baris pertama\n";
    std::cout << "Baris kedua\n";
    return 0;
}
```

Catatan penting: Indentasi tidak menentukan struktur seperti pada Python, tetapi indentasi konsisten membuat blok mudah dibaca. Hindari komentar usang yang tidak lagi cocok dengan perilaku kode.

3. Variabel dan Tipe Data

Variabel menyimpan nilai yang dapat digunakan program. Deklarasi menyebutkan tipe dan nama, misalnya `int umur{20};`. Inisialisasi dengan kurung kurawal membantu mencegah penyempitan nilai yang tidak disengaja. Tipe dasar mencakup `int` (bilangan bulat), `double` (pecahan), `char` (karakter), `bool` (benar/salah), dan `std::string` (teks).

Pilih tipe sesuai makna data. Untuk uang, pertimbangkan penyimpanan satuan terkecil sebagai integer agar tidak terkena galat pecahan biner. Gunakan `const` untuk nilai yang tidak boleh berubah setelah dibuat.

Contoh kode

```
#include <iostream>
#include <string>

int main() {
    const std::string nama{"Rani"};
    int umur{19};
    double tinggi{1.62};
    bool aktif{true};
    std::cout << nama << ", " << umur << " tahun; "
              << tinggi << " m; aktif=" << aktif << '\n';
}
```

Catatan penting: Nama variabel sebaiknya menjelaskan isi, diawali huruf atau garis bawah, dan menggunakan gaya konsisten seperti `jumlahBarang`. Hindari nama yang membingungkan atau kata kunci C++.

4. Input dan Output

`std::cout` mengirim keluaran ke terminal. Operator `<<` menyisipkan teks atau nilai ke aliran keluaran. `std::cin` membaca token yang dipisahkan spasi melalui operator `>>`. Untuk membaca seluruh baris termasuk spasi, gunakan `std::getline`.

Campuran `cin` dan `getline` dapat meninggalkan karakter newline di buffer. Setelah pembacaan dengan `cin`, gunakan `std::getline(std::cin >> std::ws, teks)` agar whitespace awal dibuang. Selalu pertimbangkan input yang tidak valid.

Contoh kode

```
#include <iostream>
#include <string>

int main() {
    std::string nama;
    int umur{};
    std::cout << "Nama lengkap: ";
    std::getline(std::cin, nama);
    std::cout << "Umur: ";
    if (!(std::cin >> umur)) {
        std::cerr << "Umur harus berupa angka.\n";
        return 1;
    }
    std::cout << "Halo, " << nama << " (" << umur << ")\n";
}
```

Catatan penting: `std::cerr` cocok untuk pesan kesalahan. Keluaran dan input dapat dirangkai; jangan menganggap masukan selalu benar—validasi sebelum menggunakannya.

5. Operator Aritmetika

Operator aritmetika dasar adalah +, -, *, /, dan %. Modulo % menghasilkan sisa pembagian bilangan bulat. Pembagian dua integer menghasilkan integer sehingga bagian pecahan terpotong; ubah salah satu operand menjadi double untuk hasil pecahan.

Operator ++ dan -- menaikkan atau menurunkan nilai satu. Bentuk awalan (++x) mengubah lalu menghasilkan nilai, bentuk akhiran (x++) menghasilkan nilai lama lalu mengubah. Hindari menyisipkan efek samping rumit dalam satu ekspresi.

Contoh kode

```
#include <iostream>

int main() {
    int a{17}, b{5};
    std::cout << a + b << '\n'; // 22
    std::cout << a / b << '\n'; // 3 (integer)
    std::cout << a % b << '\n'; // 2
    double rasio = static_cast<double>(a) / b;
    std::cout << rasio << '\n'; // 3.4
}
```

Catatan penting: Pembagian dengan nol harus dihindari. Operator precedence menentukan urutan evaluasi; gunakan tanda kurung untuk menegaskan maksud, misalnya (a + b) * c.

6. Operator Perbandingan dan Logika

Operator perbandingan menghasilkan bool: == sama dengan, != tidak sama, < lebih kecil, > lebih besar, <= dan >=. Operator logika adalah && (dan), || (atau), dan ! (negasi).

Gunakan == untuk membandingkan, bukan = yang merupakan penugasan. && dan || melakukan short-circuit: operand kanan hanya dievaluasi bila diperlukan. Ini berguna untuk menjaga pemeriksaan aman, seperti memastikan indeks valid sebelum mengakses elemen.

Contoh kode

```
#include <iostream>

int main() {
    int umur{21};
    bool punyaIzin{true};
    bool bolehMasuk = (umur >= 18) && punyaIzin;
    if (bolehMasuk) {
        std::cout << "Akses diberikan\n";
    }
    std::cout << std::boolalpha << (umur != 0) << '\n';
}
```

Catatan penting: std::boolalpha membuat bool dicetak sebagai true/false. Hindari membandingkan nilai pecahan dengan kesamaan persis; gunakan toleransi jika konteks numerik memerlukannya.

7. Penugasan dan Operator Lain

Operator penugasan = menyimpan nilai. Bentuk gabungan seperti +=, -=, *=, /=, dan %= melakukan operasi lalu menaruh hasilnya kembali. Operator kondisional ?: memilih salah satu dari dua nilai berdasarkan kondisi.

Operator bitwise &, |, ^, ~, <<, dan >> bekerja pada representasi bit integer; jangan tertukar dengan && dan ||. Operator sizeof memberi ukuran tipe/objek dalam byte. Gunakan operator bitwise hanya saat memang mengolah flag atau data bit.

Contoh kode

```
#include <iostream>

int main() {
    int skor{10};
    skor += 5; // setara skor = skor + 5
    const char* status = skor >= 12 ? "lulus" : "ulang";
    unsigned izin{0b0101};
    bool dapatBaca = (izin & 0b0001) != 0;
    std::cout << status << ", baca=" << dapatBaca << '\n';
}
```

Catatan penting: Pada contoh, bit mask memeriksa flag tertentu. Nilai bertipe unsigned cocok untuk operasi bit. Hindari mengandalkan perilaku shift yang tidak sesuai aturan tipe atau menggeser terlalu jauh.

8. Percabangan if, else, dan switch

Gunakan if untuk memilih alur berdasarkan kondisi. else if menangani kondisi tambahan dan else menjadi pilihan terakhir. Untuk pilihan diskret berdasarkan satu nilai integral atau enum, switch dapat lebih jelas.

Setiap case pada switch perlu break agar tidak berlanjut ke case berikutnya (fall-through), kecuali perilaku itu memang disengaja. Sertakan default untuk nilai yang tidak dikenal. Gunakan kurung kurawal pada blok untuk menjaga keterbacaan.

Contoh kode

```
#include <iostream>

int main() {
    int nilai{83};
    if (nilai >= 85) std::cout << "A\n";
    else if (nilai >= 70) std::cout << "B\n";
    else std::cout << "Perlu latihan\n";

    int menu{2};
    switch (menu) {
        case 1: std::cout << "Tambah\n"; break;
        case 2: std::cout << "Lihat\n"; break;
        default: std::cout << "Pilihan tidak ada\n";
    }
}
```

Catatan penting: Urutkan rentang nilai dengan benar agar batasnya tidak tumpang tindih. Jika kondisi menjadi kompleks, pecah menjadi bool bernama atau fungsi kecil.

9. Perulangan while dan do-while

Perulangan mengulang pekerjaan. while memeriksa kondisi sebelum setiap putaran, sehingga badan bisa tidak dijalankan sama sekali. do-while menjalankan badan minimal sekali dan memeriksa kondisi di akhir.

Pastikan kondisi akhirnya berubah agar tidak terjadi infinite loop. break keluar dari perulangan terdekat; continue melewati sisa badan dan melanjutkan iterasi berikutnya. Gunakan keduanya dengan hemat supaya alur tetap mudah ditelusuri.

Contoh kode

```
#include <iostream>

int main() {
    int hitung{1};
    while (hitung <= 3) {
        std::cout << hitung << ' ';
        ++hitung;
    }
    std::cout << '\n';
    int tebakkan{};
    do {
        std::cout << "Masukkan angka 1-5: ";
        std::cin >> tebakkan;
    } while (tebakkan < 1 || tebakkan > 5);
}
```

Catatan penting: Validasi input berulang perlu memperhatikan keadaan stream jika pengguna mengetik tipe yang salah. Untuk program nyata, pulihkan stream atau gunakan strategi pembacaan baris dan parsing.

10. Perulangan for dan Range-based for

for klasik menyatukan inisialisasi, kondisi, dan langkah iterasi, sehingga cocok untuk penghitung. Range-based for membaca setiap elemen koleksi tanpa mengelola indeks secara manual. Gunakan referensi jika ingin mengubah elemen atau menghindari penyalinan.

Gunakan `const auto&` untuk membaca elemen tanpa mengubahnya dan tanpa membuat salinan. Gunakan `auto&` jika memang perlu mengubah elemen. Hindari indeks di luar rentang koleksi.

Contoh kode

```
#include <iostream>
#include <vector>

int main() {
    for (int i{0}; i < 4; ++i) {
        std::cout << i << ' ';
    }
    std::cout << '\n';
    std::vector<int> angka{2, 4, 6};
    int jumlah{};
    for (const auto& n : angka) jumlah += n;
    std::cout << "Jumlah=" << jumlah << '\n';
}
```

Catatan penting: Range-based for tersedia untuk array dan koleksi standar. Header `vector` memberi kontainer dinamis yang ukurannya dapat berubah selama program berjalan.

11. Fungsi dan Parameter

Fungsi memecah program menjadi bagian yang dapat digunakan ulang. Deklarasi (prototype) menyatakan nama, tipe hasil, dan parameter; definisi berisi badan fungsi. Parameter menerima data, sedangkan return mengirim hasil kembali.

Parameter nilai mendapat salinan; perubahan lokal tidak mengubah variabel pemanggil. Referensi (&) memungkinkan fungsi bekerja pada objek asli. Tambahkan const pada referensi yang hanya dibaca. Jaga fungsi fokus pada satu tujuan.

Contoh kode

```
#include <iostream>

int tambah(int a, int b) {
    return a + b;
}

void gandakan(int& nilai) {
    nilai *= 2;
}

int main() {
    int x{7};
    std::cout << tambah(x, 3) << '\n';
    gandakan(x);
    std::cout << x << '\n';
}
```

Catatan penting: void berarti fungsi tidak mengembalikan nilai. Nama fungsi sebaiknya berupa kata kerja yang menjelaskan aksi, misalnya hitungTotal atau tampilkanMenu.

12. Cakupan, const, dan auto

Cakupan menentukan tempat nama dapat digunakan. Variabel lokal hidup di dalam blok tempat ia dideklarasikan; variabel global dapat diakses luas dan sebaiknya dihindari bila tidak diperlukan. Nama lokal di blok dalam dapat menutupi nama luar, sehingga jangan membuatnya membingungkan.

const mencegah perubahan melalui nama tersebut setelah inisialisasi. auto meminta compiler menyimpulkan tipe dari nilai awal; gunakan ketika tipe jelas dari konteks, bukan untuk menyamarkan maksud. constexpr menyatakan nilai yang dapat dihitung saat kompilasi jika syarat terpenuhi.

Contoh kode

```
#include <iostream>

int main() {
    const double pi{3.1415926535};
    constexpr int sisi{4};
    auto luasPersegi = sisi * sisi; // int
    double jari{2.0};
    double luasLingkaran = pi * jari * jari;
    std::cout << luasPersegi << ", " << luasLingkaran << '\n';
}
```

Catatan penting: Pilih tipe eksplisit saat itu memperjelas kontrak, misalnya pada antarmuka fungsi. const adalah alat penting untuk mencegah perubahan tak sengaja dan menyatakan niat.

13. Array dan std::array

Array menyimpan sekumpulan elemen bertipe sama. Array C bergaya lama memiliki ukuran tetap dan tidak menyimpan pemeriksaan batas. Untuk kode modern, std::array menggabungkan ukuran tetap dengan antarmuka kontainer yang lebih aman, termasuk .size() dan pemeriksaan .at().

Indeks dimulai dari 0, sehingga elemen terakhir berindeks size()-1. Akses di luar batas dengan operator [] dapat menghasilkan perilaku tak terdefinisi. Gunakan .at() bila pemeriksaan batas saat runtime diinginkan.

Contoh kode

```
#include <array>
#include <iostream>

int main() {
    std::array<int, 4> suhu{18, 20, 23, 21};
    int total{};
    for (int nilai : suhu) total += nilai;
    double rata = static_cast<double>(total) / suhu.size();
    std::cout << "Elemen: " << suhu.size()
              << ", rata-rata: " << rata << '\n';
    std::cout << suhu.at(0) << '\n';
}
```

Catatan penting: std::array perlu header <array>. Ukurannya menjadi bagian tipe, dan cocok ketika jumlah elemen telah diketahui saat kompilasi.

14. std::vector dan String

std::vector adalah urutan dinamis yang dapat tumbuh. Fungsi push_back menambah elemen, size membaca jumlah elemen, dan at melakukan pemeriksaan indeks. vector biasanya menjadi pilihan awal untuk daftar homogen.

std::string mewakili teks dan menyediakan operasi seperti +=, size(), dan substr(). String C++ modern mengelola memorinya sendiri. Indeks string berkaitan dengan unit char, bukan selalu satu karakter Unicode yang terlihat.

Contoh kode

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> tugas;
    tugas.push_back("Belajar tipe data");
    tugas.push_back("Latihan loop");
    for (std::size_t i = 0; i < tugas.size(); ++i)
        std::cout << i + 1 << ". " << tugas.at(i) << '\n';
    std::string judul{"C++"};
    judul += " Dasar";
    std::cout << judul << " (" << judul.size() << ")\n";
}
```

Catatan penting: Untuk daftar ukuran berubah, vector lebih aman dan nyaman daripada mengelola alokasi array mentah. Perhatikan bahwa penambahan elemen dapat memindahkan lokasi internal, sehingga pointer/referensi lama mungkin tidak lagi valid.

15. Enum dan struct

enum class mendefinisikan himpunan pilihan bernama dan bertipe kuat. Ia membantu mengganti angka ajaib dengan nama bermakna. struct mengelompokkan beberapa data terkait dalam satu tipe; anggota struct secara default bersifat publik.

Gunakan enum class agar nilai tidak tercampur otomatis dengan integer atau enum lain. Struct cocok untuk merepresentasikan data sederhana; fungsi yang mengolahnya dapat ditempatkan terpisah atau sebagai anggota class bila perilaku dan data erat terkait.

Contoh kode

```
#include <iostream>
#include <string>

enum class Status { Baru, Diproses, Selesai };
struct Produk {
    std::string nama;
    int stok;
};

int main() {
    Produk p{"Pensil", 12};
    Status status{Status::Baru};
    std::cout << p.nama << ": " << p.stok << '\n';
    if (status == Status::Baru) std::cout << "Belum diproses\n";
}
```

Catatan penting: Tipe buatan sendiri membuat data lebih terstruktur dan API lebih mudah dipahami. Pastikan setiap anggota memiliki nilai awal yang masuk akal.

16. Pengenalan Class dan Enkapsulasi

Class menggabungkan data dan fungsi serta memungkinkan aturan akses. Bagian private hanya dapat diakses oleh anggota class; public menjadi antarmuka pemakai. Enkapsulasi menjaga keadaan objek tetap valid melalui operasi yang dikendalikan.

Constructor menyiapkan objek saat dibuat. Metode const menjanjikan tidak mengubah keadaan objek. Hindari membuka seluruh data sebagai public jika ada aturan yang harus dijaga. Beri nama metode berdasarkan tindakan atau pertanyaan yang jelas.

Contoh kode

```
#include <iostream>
#include <string>

class Rekening {
    std::string pemilik;
    int saldo;
public:
    Rekening(std::string nama, int awal)
        : pemilik{nama}, saldo{awal < 0 ? 0 : awal} {}
    void setor(int jumlah) { if (jumlah > 0) saldo += jumlah; }
    int lihatSaldo() const { return saldo; }
};

int main() {
    Rekening r{"Dina", 100};
    r.setor(50);
    std::cout << r.lihatSaldo() << '\n';
}
```

Catatan penting: Daftar inisialisasi constructor (setelah titik dua) menginisialisasi anggota secara langsung. Contoh saldo memakai int untuk sederhana; sistem uang nyata sebaiknya menentukan satuan dan batas dengan cermat.

17. Pointer, Referensi, dan nullptr

Referensi adalah alias ke objek yang sudah ada dan harus diinisialisasi ketika dibuat. Pointer menyimpan alamat dan dapat bernilai nullptr. Operator & mengambil alamat; * melakukan dereferensi pointer. Dereferensi nullptr atau pointer tidak valid berbahaya.

Utamakan referensi untuk parameter wajib dan pointer hanya bila ketiadaan objek merupakan kondisi yang bermakna. C++ modern menyediakan smart pointer untuk kepemilikan dinamis; hindari new/delete manual dalam kode aplikasi pemula.

Contoh kode

```
#include <iostream>

void tambahSatu(int& x) { ++x; }
void cetakJikaAda(const int* p) {
    if (p != nullptr) std::cout << *p << '\n';
}

int main() {
    int angka{9};
    int* alamat{&angka};
    tambahSatu(angka);
    cetakJikaAda(alamat);
    alamat = nullptr;
    cetakJikaAda(alamat); // aman: tidak dereference
}
```

Catatan penting: Referensi dan pointer tidak otomatis mengelola umur objek. Pastikan objek tetap hidup selama alamat atau referensinya digunakan; jangan mengembalikan alamat variabel lokal.

18. Memori Dinamis dan Smart Pointer

Memori dinamis berguna saat umur atau jumlah objek tidak cocok dengan variabel lokal biasa. Namun kepemilikan manual menimbulkan risiko kebocoran dan penghapusan ganda. RAII mengikat sumber daya pada objek sehingga pembersihan dilakukan otomatis saat objek keluar dari cakupan.

Gunakan `std::unique_ptr` untuk kepemilikan tunggal dan `std::make_unique` untuk membuatnya. Gunakan `shared_ptr` hanya bila kepemilikan bersama benar-benar diperlukan; siklus `shared_ptr` dapat menyebabkan kebocoran. Untuk kumpulan elemen biasa, pilih `vector`, bukan alokasi manual.

Contoh kode

```
#include <iostream>
#include <memory>

int main() {
    auto angka = std::make_unique<int>(42);
    std::cout << *angka << '\n';
    // Memori dibebaskan otomatis saat angka keluar cakupan.
}
```

Catatan penting: Smart pointer berada di header `<memory>`. Jangan menyalin `unique_ptr`; kepemilikannya dapat dipindahkan dengan `std::move`. Pemahaman kepemilikan penting untuk program yang andal.

19. Algoritma STL dan Lambda

Standard Template Library menyediakan kontainer, iterator, dan algoritma siap pakai. Algoritma seperti `sort`, `find`, dan `accumulate` mengurangi kode perulangan khusus dan telah diuji luas. Iterator rentang biasanya ditulis `begin/end`; ujung `end` menunjuk posisi setelah elemen terakhir.

Lambda adalah fungsi ringkas yang dapat ditulis di tempat. Tanda kurung siku menentukan data luar yang ditangkap. Tentukan apakah nilai ditangkap salinan atau referensi; jangan menyimpan lambda yang mereferensikan variabel yang sudah mati.

Contoh kode

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nilai{8, 3, 6, 1};
    std::sort(nilai.begin(), nilai.end());
    auto genap = std::count_if(nilai.begin(), nilai.end(),
        [](int x) { return x % 2 == 0; });
    for (int x : nilai) std::cout << x << ' ';
    std::cout << "\nJumlah genap: " << genap << '\n';
}
```

Catatan penting: Algoritma memerlukan header terkait—di sini `<algorithm>`. Memakai algoritma standar sering membuat maksud lebih terbaca dibanding menulis indeks dan kondisi sendiri.

20. File I/O

Header `fstream` menyediakan `ifstream` untuk membaca, `ofstream` untuk menulis, dan `fstream` untuk keduanya. Periksa apakah file berhasil dibuka sebelum memprosesnya. Mode default `ofstream` biasanya menimpa isi file; gunakan `std::ios::app` untuk menambahkan di akhir.

Untuk teks sederhana, `getline` membaca baris. Format CSV memiliki aturan kutip dan koma yang perlu ditangani jika data rumit; jangan menganggap pemisahan dengan koma sederhana selalu cukup. Penanganan file gagal merupakan bagian penting program.

Contoh kode

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    {
        std::ofstream out("catatan.txt");
        if (!out) { std::cerr << "Gagal menulis\n"; return 1; }
        out << "Belajar C++\n";
    }
    std::ifstream in("catatan.txt");
    std::string baris;
    if (!in) { std::cerr << "Gagal membaca\n"; return 1; }
    while (std::getline(in, baris)) std::cout << baris << '\n';
}
```

Catatan penting: Berkas ditutup otomatis ketika objek stream keluar cakupan. Untuk program yang perlu berjalan di banyak sistem, pikirkan lokasi file, izin akses, encoding, dan format penyimpanan.

21. Penanganan Kesalahan dan Exception

Kesalahan dapat dilaporkan melalui nilai status, objek hasil, atau exception. Exception dengan try/catch berguna ketika operasi gagal dan pemanggil harus diberi kesempatan menangani kegagalan. Jangan gunakan exception sebagai pengganti alur normal.

Gunakan tipe exception standar yang sesuai, seperti `std::runtime_error`. Tangkap berdasarkan referensi const agar tidak menyalin objek dan tidak memotong tipe turunan. Validasi data dekat dengan batas masuk, dan jelaskan pesan kesalahan yang dapat ditindaklanjuti.

Contoh kode

```
#include <iostream>
#include <stdexcept>

int bagi(int a, int b) {
    if (b == 0) throw std::invalid_argument("Pembagi tidak boleh nol");
    return a / b;
}

int main() {
    try {
        std::cout << bagi(10, 2) << '\n';
        std::cout << bagi(10, 0) << '\n';
    } catch (const std::exception& e) {
        std::cerr << "Kesalahan: " << e.what() << '\n';
    }
}
```

Catatan penting: Exception yang tidak ditangani dapat menghentikan program. Tentukan kontrak fungsi: kondisi apa yang valid, bagaimana kegagalan dilaporkan, dan siapa yang bertanggung jawab menanganinya.

22. Header, File Terpisah, dan Namespace

Program dapat dibagi menjadi header (.hpp) dan source (.cpp). Header menyatakan antarmuka; source memuat implementasi. Include guard atau #pragma once mencegah header diproses berulang. Namespace mengelompokkan nama dan mengurangi benturan nama.

Jangan menulis using namespace std; di header karena efeknya menyebar ke semua berkas pemakai. Sertakan header yang benar-benar dibutuhkan. Saat membangun proyek kecil, kompilasikan seluruh file sumber bersama atau gunakan sistem build seperti CMake.

Contoh kode

```
// math_utils.hpp
#pragma once
int kuadrat(int nilai);

// math_utils.cpp
#include "math_utils.hpp"
int kuadrat(int nilai) { return nilai * nilai; }

// main.cpp
#include <iostream>
#include "math_utils.hpp"
int main() { std::cout << kuadrat(6) << '\n'; }
// Build: g++ -std=c++17 main.cpp math_utils.cpp -o app
```

Catatan penting: Dalam proyek nyata, deklarasi umumnya berada di namespace proyek sendiri, misalnya namespace aplikasi. Memisahkan antarmuka dari implementasi mempercepat pemahaman dan pemeliharaan.

23. Template dan Pemrograman Generik

Template memungkinkan fungsi atau class bekerja dengan berbagai tipe tanpa menyalin implementasi. Compiler membuat versi yang diperlukan berdasarkan tipe pemakaian. Parameter template dapat berupa tipe maupun nilai compile-time. Kesalahan template kadang panjang, jadi mulai dari contoh kecil.

Gunakan template saat operasi benar-benar berlaku pada beberapa tipe dan kontraknya jelas. Untuk fungsi generik sederhana, parameter dapat disimpulkan otomatis. Template umumnya didefinisikan di header agar compiler melihat implementasinya saat instansiasi.

Contoh kode

```
#include <iostream>

template <typename T>
T maksimum(T a, T b) {
    return (a < b) ? b : a;
}

int main() {
    std::cout << maksimum(4, 9) << '\n';
    std::cout << maksimum(2.5, 1.8) << '\n';
}
```

Catatan penting: Kedua argumen pada contoh harus memiliki tipe yang sama agar T dapat disimpulkan. Template C++ modern juga mendukung batasan konsep (C++20) untuk menyatakan operasi yang disyaratkan.

25. Proyek Mini: Pengelola Nilai

Gabungkan konsep dasar dalam program kecil: vector menyimpan nilai, fungsi menghitung rata-rata, input divalidasi, dan perulangan menampilkan ringkasan. Pecah pekerjaan menjadi langkah: baca jumlah data, kumpulkan nilai, hitung hasil, lalu tampilkan. Pengembangan kode juga perlu praktik baik: nama jelas, peringatan compiler aktif, dan pengujian kasus normal, batas, serta input tidak valid.

Contoh berikut memakai angka nilai bulat sederhana. Periksa kegagalan input dan pastikan jumlah data positif. Setelah berhasil, kembangkan dengan penyimpanan file, nilai minimum/maksimum, atau pemisahan fungsi ke beberapa berkas.

Contoh kode

```
#include <iostream>
#include <vector>

double rataRata(const std::vector<int>& data) {
    int total{};
    for (int n : data) total += n;
    return static_cast<double>(total) / data.size();
}

int main() {
    int jumlah{};
    std::cout << "Jumlah nilai: ";
    if (!(std::cin >> jumlah) || jumlah <= 0) return 1;
    std::vector<int> nilai;
    for (int i = 0; i < jumlah; ++i) {
        int n{};
        std::cout << "Nilai " << i + 1 << ": ";
        if (!(std::cin >> n) || n < 0 || n > 100) return 1;
        nilai.push_back(n);
    }
    std::cout << "Rata-rata: " << rataRata(nilai) << '\n';
}
```

Catatan penting: Ringkasan belajar: pahami tipe, operator, kendali alur, fungsi, kontainer, dan validasi input. Latihan terbaik adalah mengubah program sedikit demi sedikit, mengompilasi, menguji kasus batas, serta membaca dokumentasi pustaka standar. Untuk pengujian, jalankan beberapa kombinasi input dan periksa hasil yang diharapkan; assert dapat membantu memeriksa asumsi selama pengembangan.

GABUNG KOMUNITAS THEPORT COMMUNITY

Belajar dan berdiskusi bersama komunitas Theport melalui WhatsApp:

<https://chat.whatsapp.com/CQvGHUPIBrFCv9QCqzg0bb>

Diterbitkan oleh Theport Studio

Pendiri: Farel Alfareza

Admin lainnya:

Evan · Daffa · Revigata